

Finance Tracker Application

Final Project Report

Author: Ethan Hartman

Date: August 2025

Project Overview

The Finance Tracker application was conceived as a response to the common challenges individuals and small organizations face when trying to manage their financial data. Spreadsheets and off-the-shelf apps often provide only surface-level solutions — they either lack customization, store data in fragile formats, or come with hidden costs and subscription models. The Finance Tracker was designed to fill that gap by offering a flexible, configurable, and open-ended platform for financial management.

At its core, the Finance Tracker provides users with the ability to:

- **Track Accounts and Balances:** Multiple accounts can be created and managed simultaneously, each tied to unique identifiers and transaction histories.
- **Record and Categorize Transactions:** Transactions may be logged as credits or debits, accompanied by descriptive notes and optional category tags.
- **Budget Effectively:** Users can define monthly or category-specific budgets, enabling proactive financial planning rather than reactive expense tracking.
- **Set and Monitor Goals:** Savings or investment goals are built directly into the system, with mechanisms for measuring progress.
- **Receive Alerts and Insights:** Budget overruns or missed goals trigger alerts, helping users course-correct in real time.

These features were intentionally designed to be minimal but extensible. The first release focuses on stability and reliability, while the architecture leaves room for integration with advanced analytics, machine learning predictions, and user interface upgrades.

Purpose and Motivation

The motivation behind the Finance Tracker project stemmed from two primary considerations:

1. **Educational Value:** The project serves as a hands-on implementation of modern software engineering principles. It demonstrates the practical use of MVC

architecture, ORM modeling, Agile Unified Process, and testing methodologies in a real-world scenario.

2. **Practical Utility:** Beyond being an academic exercise, the application addresses real-world needs. It equips users with tools to track daily finances without reliance on proprietary cloud platforms or third-party apps that often compromise on privacy and flexibility.

The vision was to build an application that could serve as both a teaching tool for developers and a practical finance manager for end users.

Scope

The project was scoped with a balance between ambition and feasibility. The system was designed to handle the fundamental tasks of personal finance management, but within a bounded architecture that allows for natural iteration. Its current scope includes:

- Account and Transaction Management (core functionality).
- Budget Creation and Monitoring.
- Goal Setting and Tracking.
- Alerts for Overspending.
- Command-Line Interface for interaction.

The system deliberately excludes features such as bank synchronization, multi-user authentication, or graphical visualization in its initial release. However, the architecture anticipates these needs and makes them feasible for future development without large-scale refactoring.

Architecture Choices

The application employs a Model–View–Controller (MVC) architecture to ensure clear separation of concerns. SQLAlchemy ORM was chosen for database management due to its flexibility and strong ecosystem support. Alembic provides controlled schema migrations, allowing the project to evolve without losing historical data integrity.

The decision to use Python 3.13 reflects a commitment to staying current with language features and performance improvements. A TOML-based configuration system ensures that the application can adapt easily to different environments (development, testing, and production), reinforcing good engineering practices such as config-driven development.

Intended Impact

The Finance Tracker aims to deliver value on multiple levels:

- For End Users: A reliable tool to improve financial awareness and decision-making. By centralizing records and automating tasks such as budget alerts, the system encourages users to stay on top of their money rather than reacting to problems after they arise.
 - For Developers: A reference project showcasing how to apply Agile Unified Process, UML modeling, and test-driven design to a practical problem domain. The modular architecture also allows developers to experiment with extensions, such as machine learning-based financial forecasting.
 - For the Project Owner: A concrete demonstration of the ability to plan, design, implement, and document a real-world software system, balancing both technical and user-centered perspectives.
-

System Design and UML Models

The Finance Tracker application was designed with an emphasis on clarity, modularity, and extensibility. To achieve this, the development adopted a Model–View–Controller (MVC) style architecture, separating the data layer, the business logic, and the user interaction components. This structure ensures that changes in one layer do not propagate unintended consequences into other layers, making the system easier to maintain and extend over time.

Architectural Principles

1. Low Coupling, High Cohesion:
Each module was designed with a single, well-defined responsibility. For example, the models strictly handle database persistence, while controllers deal with logic such as transaction filtering and validation. This makes it easier to extend functionality without rewriting large parts of the system.
2. Configuration-Driven Development:
All environment settings, including database URLs, logging verbosity, and echo settings, are defined in a `config.toml` file. This allows seamless switching between development, testing, and production environments without modifying the source code. It also supports portability, ensuring that different developers or deployments can use their own configurations.

3. Database Abstraction with ORM:

The system uses SQLAlchemy ORM to abstract database interactions. This choice provides both flexibility (supporting multiple relational databases) and safety (automatic SQL generation, type enforcement, and relationship management). Schema evolution is managed through Alembic, ensuring that migrations can be tracked and reproduced consistently across environments.

4. Iterative Extensibility:

Although the initial release uses a command-line interface (CLI) for user interaction, the architecture anticipates upgrades to a graphical user interface (GUI) or web application. Because views are abstracted from controllers, introducing a new presentation layer will not require rewriting business logic.

Core Components

1. Models (Persistence Layer):

- Account: Represents financial accounts (e.g., checking, savings, business). Tracks balances and transaction associations.
- Transaction: Stores credits and debits with fields for date, amount, description, and category. Each transaction belongs to an account.
- Budget: Defines spending limits by category or timeframe. Triggers warnings when thresholds are approached or exceeded.
- Goal: Represents savings or financial objectives, with progress tracked against transactions and account balances.
- Alert: Encapsulates system warnings, such as budget overruns or goal shortfalls.

Each of these models is defined using SQLAlchemy classes, with relationships (one-to-many, many-to-one) linking them into a coherent schema. For instance, an account may contain many transactions, but each transaction belongs to a single account.

2. Controllers (Business Logic Layer):

- Handle transaction validation (ensuring a CREDIT increases balance while a DEBIT decreases it).
- Provide filtering mechanisms (by date range, transaction type, category, or account).

- Enforce budget and goal rules, raising alerts when conditions are met.
 - Serve as an intermediary between the models and the presentation layer, ensuring consistent application of business rules.
3. Views (Presentation Layer):
- Currently CLI-driven, the views prompt users for input and display data in readable summaries.
 - Designed so that they could be replaced by graphical widgets, web pages, or even API endpoints without altering controllers or models.
4. Configuration and Database Modules:
- config/: Loads TOML configuration files and supplies environment-specific variables to the rest of the system.
 - db/: Manages the database engine, sessions, and connections. Serves as the foundation for all ORM interactions.
-

UML Models

The Finance Tracker project includes multiple UML diagrams to illustrate its structure and operation. These diagrams provide both high-level architectural overviews and detailed interaction models.

1. Class Diagram:

Displays the static structure of the system, focusing on the relationships among entities.

- Account aggregates multiple Transaction objects.
- Transaction is linked to Category and belongs to one Account.
- Budget references categories and enforces spending rules.
- Goal relates to Account balances to measure savings progress.

This diagram highlights the relational structure that the ORM enforces at the database level.

2. Sequence Diagram:

Captures the dynamic behavior of the system when a user enters a new transaction.

- The user interacts with the CLI view to enter transaction details.

- The controller validates the input and ensures it adheres to system rules.
- The model persists the data into the database, updating account balances.
- If applicable, alerts are triggered (e.g., overspending warning).

This diagram emphasizes the flow of control across layers and reinforces the separation of responsibilities.

3. Component Diagram:

Provides a high-level view of the major modules (models/, controllers/, views/, config/, db/).

- Arrows indicate dependencies (e.g., controllers depend on models, views depend on controllers).
- The database is treated as an external component abstracted by SQLAlchemy.

This diagram demonstrates modularity and how the system could be extended in the future.

Rationale for Design Choices

- MVC Pattern: Chosen for its clarity, testability, and widespread adoption. It makes the system easy to reason about and extend.
- SQLAlchemy ORM: Provides abstraction and eliminates boilerplate SQL. It allows the same code to run on different databases (SQLite for development, PostgreSQL or MySQL for production).
- Alembic: Ensures database schema changes can be versioned and applied consistently, a critical feature for long-term maintainability.
- TOML Configurations: TOML is human-readable, supports comments, and is widely used in Python tooling, making it an ideal choice for config-driven environments.
- CLI First: A command-line interface simplifies initial development and testing. By avoiding early complexity in UI development, the project focused resources on building a strong backend.

Benefits of the Design

- **Maintainability:** Clear separation of concerns ensures that changes can be localized to a single layer.
 - **Extensibility:** The architecture anticipates new features, such as GUIs, mobile apps, or financial forecasting engines.
 - **Portability:** Configuration-driven design and ORM abstraction make the application portable across systems and databases.
 - **Reliability:** Strong typing, migrations, and testing ensure that data integrity is preserved.
-

Development Process (Agile/Unified Process Implementation)

The development of the Finance Tracker application followed a disciplined yet flexible methodology, drawing heavily from the Agile Unified Process (AUP). The AUP was selected because it combines the rigor of the traditional Rational Unified Process (RUP) with the adaptability of Agile methodologies. This provided a structure for managing phases such as inception and elaboration, while still enabling iterative progress, continuous feedback, and rapid adjustment to challenges.

Rationale for Agile Unified Process

Pure Agile methods such as Scrum or Kanban emphasize speed and adaptability, but can sometimes lack the architectural foresight needed for complex systems. Conversely, heavyweight methodologies like Waterfall or full RUP can overemphasize documentation and planning at the expense of adaptability. The AUP was chosen as a middle ground, ensuring:

- A clear architectural vision.
- Incremental delivery of working software.
- Continuous refinement through iterations.
- Documentation that is useful, but not excessive.

This balance was ideal for the Finance Tracker, which needed both solid architectural foundations (e.g., ORM schema, migrations) and the flexibility to incorporate lessons learned during implementation.

Process Phases

1. Inception Phase:

The inception phase defined the scope, vision, and constraints of the project. Key deliverables included:

- Project objectives (personal finance management, modular design).
- Technology stack selection (Python 3.13, SQLAlchemy, Alembic, TOML).
- Initial risk assessment (ORM complexity, migration handling, CLI usability).
- Draft UML diagrams to visualize potential system structure.

One of the critical decisions made during this phase was to adopt the MVC architecture from the outset. This ensured that even as the project grew, it would remain organized and extensible.

2. Elaboration Phase:

In this phase, the project moved from abstract requirements to concrete architecture. Major tasks included:

- Establishing the src project layout, avoiding flat-layout errors and aligning with Python packaging best practices.
- Setting up the Alembic environment for migrations.
- Designing the first version of the database schema (accounts, transactions, budgets, goals).
- Creating the configuration loader (config.toml), supporting multiple environments (dev, test, prod).

The elaboration phase confirmed that the technical foundation was sound before moving into implementation.

3. Construction Phase:

The construction phase was iterative, broken into sprints, each focusing on implementing core features. Deliverables were tangible, working increments of the application.

- Sprint 1:
 - Implemented base models (Account, Transaction).
 - Drafted migration scripts with Alembic.
 - Verified schema persistence using SQLite.

- Sprint 2:
 - Added Budget and Goal models.
 - Implemented transaction filtering (by type, date, category).
 - Wrote initial unit tests with pytest.
- Sprint 3:
 - Expanded CLI interface with account creation, transaction entry, and reporting.
 - Conducted integration tests to validate schema migrations.
 - Authored initial user manual draft.
- Sprint 4 (planned/future):
 - Advanced reporting features (charts, summaries).
 - Graphical or web-based UI prototypes.
 - Enhanced test coverage with regression suites.

Each sprint ended with a retrospective review, identifying what worked well (e.g., modular design choices) and what needed refinement (e.g., naming consistency in filters).

4. Transition Phase:

The final phase emphasized preparing the system for deployment and usage. Tasks included:

- Writing end-user documentation (user manual).
- Conducting walkthrough tests of the CLI interface to validate usability.
- Ensuring that the Alembic migration history was stable and reproducible.
- Packaging the project in a way that could be shared, installed, and run by others.

This phase also involved documenting lessons learned and areas for future improvement, ensuring continuity if the project is handed off or expanded later.

- **Iterative Development:** Features were implemented in small increments, tested, and integrated immediately, reducing the risk of large, late-breaking failures.
 - **Backlog Management:** Requirements were treated as a flexible backlog, allowing re-prioritization when issues like Alembic configuration errors emerged.
 - **Daily Logging (Stand-ins for Standups):** Instead of formal team standups, daily progress notes were kept to track tasks, blockers, and lessons.
 - **Retrospectives:** At the end of each sprint, lessons were captured and applied to the next sprint. For example, Sprint 1 exposed environment setup problems, which were fully resolved in Sprint 2.
-

Advantages of the Chosen Process

1. **Flexibility:** Allowed for mid-course corrections, such as restructuring the package layout.
 2. **Risk Mitigation:** Architectural decisions (e.g., ORM, migrations) were validated early, reducing the chance of late-stage surprises.
 3. **Transparency:** Frequent testing and documentation ensured the project's state was always visible.
 4. **User-Centered Design:** Usability feedback on the CLI workflows directly influenced refinements to transaction entry and reporting.
-

Development Tools and Environment

- **IntelliJ IDEA Community Edition:** Served as the IDE, offering project management and Python integration.
 - **Python 3.13 Virtual Environment:** Managed dependencies with venv to isolate development.
 - **Alembic:** Ensured reproducible and traceable schema migrations.
 - **Pytest:** Simplified test writing and execution.
 - **SQLite:** Used for both testing and development due to its speed and portability.
-

Reflection on Process

The Agile Unified Process allowed the project to balance rigor with adaptability. By combining architectural foresight with iterative feedback, the Finance Tracker was able to evolve steadily without accumulating significant technical debt.

Future iterations will continue to use this methodology, with greater emphasis on automated CI/CD pipelines, expanded regression testing, and possibly Scrum-inspired sprint planning if team development expands.

Testing Methodologies Used

Testing was considered a critical pillar of the Finance Tracker project, ensuring that the system remained stable, reliable, and user-friendly throughout its development lifecycle. From the very beginning, testing was integrated into the process rather than treated as an afterthought. This proactive approach reduced the likelihood of regressions, confirmed that requirements were met, and validated the correctness of both the business logic and the persistence layer.

Testing Philosophy

The Finance Tracker adopted a test-driven mindset wherever possible. While the project did not strictly follow Test-Driven Development (TDD) in every case, the guiding philosophy was that every feature must be accompanied by tests that prove it works.

The testing methodology was guided by three principles:

1. **Incremental Validation:** As features were introduced (e.g., transaction filtering, goal tracking), they were immediately paired with tests to validate correctness.
 2. **Layered Coverage:** Tests were designed to operate at different levels — from the smallest unit of code (e.g., transaction type conversion) to full workflow tests (e.g., creating an account, adding transactions, and viewing a budget summary).
 3. **Automated Repeatability:** Tests were written in pytest and used fixtures with an in-memory SQLite database. This allowed test runs to be automated and repeated, ensuring consistent results across development environments.
-

Types of Testing

1. Unit Testing

Unit tests were written to validate individual methods and functions. These focused on correctness of isolated logic, such as:

- Ensuring that transactions correctly update account balances.
- Verifying that the filter system properly interprets transaction types (CREDIT, DEBIT).
- Confirming that budget thresholds correctly trigger alerts when exceeded.

Example unit test case:

```
def test_credit_transaction_increases_balance(session, account):  
  
    tx = Transaction(account_id=account.id, amount=100,  
                      type=TransactionType.CREDIT)  
  
    session.add(tx)  
  
    session.commit()  
  
    assert account.balance == 100
```

2. Integration Testing

Integration tests ensured that the system's components worked together correctly. These were particularly important in validating Alembic migrations and ORM interactions. For example:

- Running an Alembic migration, then inserting sample data, to confirm that new schema changes did not break queries.
- Executing full workflows (e.g., creating an account, logging transactions, retrieving reports) to validate end-to-end functionality.

Integration tests provided confidence that the Finance Tracker's layers — config, ORM, and controllers — interacted as expected.

3. Regression Testing

As the schema evolved, regression testing ensured that existing functionality was not broken by new migrations or features. For example, when Budget models were added, regression tests confirmed that existing transaction workflows still behaved as intended.

4. Exploratory/Manual Testing

In addition to automated tests, manual walkthroughs of the CLI interface were

conducted. These focused on user experience and usability rather than technical correctness, identifying issues such as unclear prompts or edge-case behavior (e.g., entering invalid input for a transaction amount).

Testing Strategy

- **Database Fixtures:** Tests used an in-memory SQLite database to ensure that tests were isolated, fast, and reproducible. Each test run started with a fresh database schema created from the latest models.
 - **Fixtures and Factories:** pytest fixtures were used to generate reusable accounts, transactions, and budgets, reducing boilerplate code and ensuring consistency.
 - **Test Coverage Goals:** The goal was to maintain coverage of core modules, particularly around transaction logic and ORM operations. While full coverage was not achieved in this iteration, the foundation was established for future expansion.
 - **Continuous Validation:** Tests were run frequently during development to validate that incremental changes did not break functionality. Although a CI/CD pipeline was not fully implemented, the project is structured to easily integrate with tools such as GitHub Actions or GitLab CI.
-

Sample Test Cases and Results

1. Transaction Balance Update
 - *Input:* Add a credit of \$200 to an account with balance \$0.
 - *Expected:* New balance = \$200.
 - *Result:* Test passed.
2. Transaction Filtering by Type
 - *Input:* Query all CREDIT transactions in a dataset containing 5 credits and 3 debits.
 - *Expected:* Returned transactions = 5.
 - *Result:* Test passed.
3. Budget Threshold Alert

- *Input:* Budget set to \$500, transactions totaling \$520 logged.
- *Expected:* Alert triggered.
- *Result:* Test passed, alert correctly generated.

4. Migration Integrity

- *Action:* Run alembic upgrade head on a fresh environment, insert a transaction, and query by account.
 - *Expected:* Migration runs without errors, transaction stored successfully.
 - *Result:* Test passed.
-

Tools and Frameworks

- **pytest:** Chosen for its simple syntax, fixture system, and compatibility with CI.
 - **SQLite In-Memory Database:** Ensured rapid test execution without persisting data between runs.
 - **Alembic:** Tested migration scripts to confirm schema evolution reliability.
 - **IntelliJ IDEA Integration:** Allowed direct execution of test suites within the IDE for rapid debugging.
-

Future Testing Plans

The testing methodologies established a strong foundation but also identified opportunities for future improvement:

- **Automated Regression Testing:** As the project grows, regression suites will be automated and integrated into a CI pipeline.
- **Coverage Analysis:** Tools such as pytest-cov will be added to monitor and improve test coverage, particularly around user interface code.
- **Property-Based Testing:** Libraries like Hypothesis could generate randomized test inputs to identify edge cases beyond manually written tests.
- **Usability Testing:** Formal user testing sessions will be conducted to evaluate CLI workflows and, later, graphical interfaces.

Summary

The Finance Tracker’s testing methodologies ensured that the application was not only functional but also resilient, maintainable, and trustworthy. By combining unit, integration, regression, and manual testing, the project achieved a balance of technical correctness and user validation. These practices also laid the groundwork for scaling testing efforts as the application evolves, ensuring that future enhancements — such as GUI support, analytics, or multi-user features — can be added with confidence.

Challenges Faced and Lessons Learned

Several challenges emerged during the development process:

1. **Environment Setup:** Migrating to Python 3.13 required updates to virtual environments and SDK paths in IntelliJ.
2. **Package Layout:** The initial flat-layout caused setuptools errors. Switching to a src-layout resolved dependency issues.
3. **Alembic Configuration:** Missing database keys in config.toml caused migration failures until proper sections were added.
4. **Transaction Filters:** Confusion between txtype and type attributes highlighted the importance of consistent naming conventions.
5. **IDE Integration:** IntelliJ marked files in red when the interpreter was misconfigured, demonstrating the importance of correct development environment setup.

Lessons Learned

- Early investment in architecture and configuration avoids long-term technical debt.
 - Clear naming conventions prevent unnecessary bugs.
 - A modular structure minimizes the ripple effect of changes.
 - Tooling and IDE setup directly affect productivity and should not be overlooked.
-

User Manual – Guide for End Users

The Finance Tracker application is a lightweight but powerful tool designed to help you manage personal and small-business finances. This guide walks you through installation, configuration, day-to-day use, and troubleshooting.

1. Installation and Setup

System Requirements

- Operating System: Windows 10/11, macOS, or Linux
- Python Version: Python 3.13 or later
- Dependencies:
 - SQLAlchemy
 - Alembic
 - Pytest (optional, for running tests)

Step 1: Clone or Download

Download the project repository from GitHub (once hosted) or copy the source files.

```
git clone https://github.com/yourusername/finance-tracker.git
cd finance-tracker
```

Step 2: Create a Virtual Environment

Create and activate a Python virtual environment:

```
python3.13 -m venv .venv
source .venv/bin/activate    # macOS/Linux
.venv\Scripts\activate      # Windows
```

Step 3: Install Dependencies

```
pip install -r requirements.txt
```

Step 4: Configure the Application

The application uses a config.toml file for settings. Example:

```
[database]
url = "sqlite:///finance_tracker.db"
```


echo = true

[env]

mode = "dev"

You can change the database URL to use SQLite, PostgreSQL, or MySQL depending on your needs.

2. Launching the Application

To start the Finance Tracker, run:

```
python main.py
```

The application will launch in command-line mode, displaying a simple menu of options.

3. Core Features and Workflows

3.1 Account Management

Create an Account:

- Choose “Create Account” from the main menu.
- Provide an account name (e.g., *Checking*).
- The account is assigned a unique ID automatically.

List Accounts:

- View all available accounts, their IDs, and current balances.

Delete an Account:

- Select an account by ID, confirm deletion.
-

3.2 Recording Transactions

Transactions are the backbone of the system. Each transaction requires:

- Type: CREDIT (income) or DEBIT (expense).

- Amount: A positive numeric value.
- Description: A short text (e.g., “Groceries”).
- Category (optional): e.g., Food, Utilities, Salary.

Example Input:

Enter transaction type [credit/debit]: debit

Enter amount: 45.50

Enter description: groceries

Enter category (optional): food

System Response:

Transaction recorded successfully.

Current balance for Account 1: \$954.50

3.3 Filtering Transactions

Users can query transactions based on filters:

- By Type: Show only CREDIT or only DEBIT.
- By Date Range: From YYYY-MM-DD to YYYY-MM-DD.
- By Category: e.g., show only “Food” expenses.
- By Account: Filter by a specific account ID.

Example Command:

Filter: show all debits for Food category this month

System Output:

2025-08-01 | Debit | \$45.50 | Groceries

2025-08-05 | Debit | \$22.75 | Coffee Shop

Total: \$68.25

3.4 Budgeting

Budgets allow users to plan spending for categories or timeframes.

Create a Budget:

- Enter category (e.g., *Food*).
- Enter amount (e.g., \$300 per month).

The system will monitor expenses in that category. If spending approaches the threshold, an alert is generated.

Example:

Budget set: Food, \$300

Alert: 90% of budget reached for Food (\$270 spent).

3.5 Goals

Goals track savings or investment objectives.

Create a Goal:

- Enter description (e.g., *Emergency Fund*).
- Enter target amount (e.g., \$1000).

Progress updates dynamically based on account balances.

Example:

Goal: Emergency Fund

Target: \$1000

Progress: \$450 (45%)

3.6 Alerts

Alerts notify users about conditions such as:

- Budget exceeded.
- Goal met or lagging.
- Large debit transactions.

These appear automatically when relevant, keeping users informed without requiring manual checks.

3.7 Reports

The application can generate textual reports, such as:

- Monthly Summary: Total credits, total debits, and net change.
- Category Breakdown: Spending by category.
- Budget Adherence: Which budgets were exceeded, met, or under.

Example Report:

August 2025 Summary

Credits: \$3200

Debits: \$2750

Net Change: +\$450

Top Spending Categories:

- Rent: \$1200

- Food: \$340

- Utilities: \$200

4. Advanced Usage

Running Migrations

If schema changes occur:

```
alembic revision --autogenerate -m "add alerts table"
```

```
alembic upgrade head
```

Switching Environments

In config.toml, update:

```
[env]
```

```
mode = "prod"
```

This switches from a development database to a production database.

Backup

Simply copy the .db file (if using SQLite) or use your database's native export tool.

5. Troubleshooting

- Error: Database connection failed
 - Check config.toml for correct URL.
 - Ensure your database server is running (if not SQLite).
 - Red files in IntelliJ IDEA
 - Confirm the project is using the correct Python interpreter (.venv).
 - Migrations not running
 - Ensure [database] section exists in config.toml.
 - Transactions not saving
 - Confirm you committed changes (session.commit() is required in SQLAlchemy).
-

6. Best Practices

- Backup your database file regularly.
 - Use categories consistently to maximize reporting accuracy.
 - Set realistic budgets and goals to make alerts meaningful.
 - Start with a single account (e.g., Checking) before adding multiple.
 - Run tests periodically to confirm your installation remains healthy.
-

7. Exit Commands

At any time, type:

q

quit

exit

to leave the application.

Conclusion and Future Work

The Finance Tracker demonstrates how modern software engineering practices can be applied to personal finance management. It provides a strong architectural foundation while remaining lightweight and accessible for everyday use.

Looking forward, several enhancements are planned:

- Graphical User Interface (GUI): Replace the CLI with a more user-friendly desktop or web application.
- Analytics and Visualization: Integrate charts, graphs, and predictive financial models.
- Mobile Integration: Build a Kotlin or React Native app for mobile access.
- Security Enhancements: Encrypt stored data and add support for multiple user accounts.
- Cloud Deployment: Enable distributed usage with synchronization across devices.

By maintaining modularity and scalability, the Finance Tracker is positioned not only as a practical personal tool but also as a foundation for larger-scale financial management solutions.